Object Detection under Occlusion : CS 7643

Paing Khant, Shafeeque Qazi, Eli Smith, Chen Chen
Georgia Institute of Technology
pkhant3@gatech.edu, sqazi7@gatech.edu, eli.smith@gatech.edu, cchen334@gatech.edu

Abstract

Detecting objects in images is crucial for various real-world applications but faces significant challenges under occlusion, where objects are partially hidden. Our work focuses on understanding state-of-the-art research and demonstrating an end-to-end solution to this complex problem. We explored and compared the efficacy of different deep neural network architectures, namely Convolutional Neural Networks (CNNs) and Vision Transformers, in handling natural occlusions. We fine-tuned and evaluated pretrained Faster R-CNN, Faster R-CNN with Swin Transformer backbone, Detection Transformer (DETR), and Pix2seq models on the Dhaka Occluded Objects dataset. We assembled our pre-training dataset (OccUrban7) by merging publicly available image classification datasets that overlap with our Dhaka subjects. We tuned the dataset for use on occlusion tasks by applying an occlusive augmentation pipeline and by training a custom GAN to add a fog effect to the images. Furthermore, we designed and implemented a custom architecture, "BoT-Det," pre-training its backbone on our image classification dataset with synthetic occlusions (OccUrban7) and did transfer learning on the Dhaka dataset. Our study highlights the limitations of current standard object detection models in real-world occluded scenarios and proposes a potential solution through a specialized architecture.

1. Introduction/Background/Motivation

Detecting objects in images is an important task for many real-world applications, including autonomous driving, robotics, and medical image analysis. This task is currently performed by deep neural networks, which have achieved remarkable performance. However, most standard object detection models rely on a full, unobstructed view of objects in the images to make accurate predictions. In real-world scenarios, such ideal conditions are not always present and the challenge of occlusion exists. Occlusion occurs when a part of an object is hidden or obscured by another object in an image. We are specifically interested in natural occlusions caused by movement and interactions in the real-world. Figure 1 illustrates an example of how the bus is occluded (hidden) by an auto-rickshaw.

Figure 1: Example of occlusion in a select image from the Dhaka dataset.

In this work, we aim to explore the efficacies of different model architectures to handle occlusion and attempt to design a model that can better handle occlusions. Specifically, we aim to determine how well object detectors based on convolutional neural networks compare to their counterparts that use vision transformers. We fine-tuned and evaluated the performance of the following pretrained models: Faster R-CNN [10], Faster R-CNN with a Swin transformer [7] backbone, DEtection TRansformer (DETR) [1], and Pix2seq [2]. Based on these results, we designed our own architecture to handle occlusion. We pretrained the backbone of our architecture on an image classification dataset with synthetic occlusions by adding random patches of colored pixels. Then we integrated the backbone into the entire architecture and trained it on an occluded object detection dataset (Dhaka) to compare its performance against the baseline models.

Most current standard object detection models are not designed to handle occlusion. Current approaches to handling occlusion include data augmentation to introduce synthetic occlusions [5] and context-aware models [16], which infer the obscured parts by analyzing the visible parts [12] and surrounding context. However, most approaches still

Dataset	Task	Num Images	Num Classes	Occlusion Type
Dhaka Occluded Objects	Object detection	5080	8	Natural
Occluded Urban Images (OccUrban7)	Image classification	24600	7	Synthetic

Table 1: Description of datasets

face limitations: the performance of models degrade under heavy occlusion and many methods rely on computationally expensive architecture. Additionally, the occlusion in most datasets do not reflect the natural occlusions that occur in the real world. Therefore, we decided to explore techniques to improve the performance of object detection models at handling natural occlusion in an outdoor environment. The ability to detect occluded objects will lead to more reliable models that can be deployed in real-world environments, such as autonomous driving and robotics.

We used two datasets: Dhaka Occluded Objects (Dhaka) [9] and Occluded Urban Images (OccUrban7). Table 1 provides a brief description of the datasets we used. Dhaka was used to finetune and compare the performance of existing architectures against ours for detecting occluded objects. OccUrban7 was used to pretrain the backbone of our architecture before the entire model was trained on Dhaka.

The Dhaka Occluded Objects dataset, contained 5080 images of 8 object categories from road scenes in Dhaka, Bangladesh. The dataset contained annotations in COCO [6] format, but did not have annotations for object masks. This unfortunately led to some limitations in the data available for training and our choice of model architectures. However, this dataset was suitable since the images had plenty of natural occlusions resulting from movement and interactions of people in traffic. OccUrban7 contains images of different classes present in Dhaka with synthetic occlusions created by adding patches of colored pixels.

2. Approach

To determine whether convolutional neural networks or transformers better handle occlusions, we fine tuned and evaluated the performance of the following pretrained models on the Dhaka dataset: Faster R-CNN [10], Faster R-CNN with a Swin transformer [7] backbone, DEtection TRansformer (DETR) [1], and Pix2seq [2]. DETR and Pix2seq models are both encoder-decoder architectures that use transformers to perform object detection. DETR specifically uses a CNN backbone to extract features, which are then fed to the transformer encoder-decoder, while Pix2seq converts images into a sequence of tokens that are input directly into the encoder.

Next, we created an image classification dataset with the same classes as those in Dhaka. We will call this the Occluded Urban Images (OccUrban7) dataset. This dataset is used to pretrain the backbone of our custom architecture, called BoTDet, before training and evaluating it on Dhaka.

2.1. Pix2Seq Architecture

One of the approaches is to use the Pix2Seq[2] model to predict object bounding boxes and class labels on the Dhaka Occluded Dataset. We fine-tuned a Pix2Seq model, utilizing a DeiT (Data-efficient Image Transformer) [14] encoder, on the Dhaka Occluded Dataset. Given the poor performance observed when training directly on the dataset, we proposed pretraining the encoder on OccUrban7 dataset. Our motivation was that pretraining the encoder on related object categories could help the model better extract relevant features and improve detection under occlusion. We believed that this would enhance the model's ability to handle occlusions, given that the Pix2Seq architecture can outperform Faster R-CNN and DETR on standard object detection tasks. However, we anticipated that even with these modifications, performance might still be limited due to the extreme challenges posed by occluded scenes.

Throughout the process, we encountered several difficulties. One major challenge was that Pix2Seq models output a language-like sequence, representing object class labels and bounding box coordinates, rather than direct bounding box predictions. This necessitated additional transformation steps to evaluate the outputs using COCO-style metrics based on IoU (Intersection over Union). Furthermore, the model does not inherently guarantee that the predicted bounding boxes are properly aligned or rectangular, introducing additional noise into evaluation. As anticipated, the initial fine-tuning attempts yielded very low mAP scores, confirming the difficulty of the task and validating our decision to pursue a more customized encoder training strategy. The project required multiple rounds of hyperparameter tuning and adjustment before we observed any measurable improvement in detection performance.

2.2. Pre-Training Motivation

We also pre-trained the backbone of our model BoTDet with an image classification task to narrow-down their domain to images of objects that are more relevant to the Dhaka task. Most backbone networks are often pre-trained on very large/general datasets like ImageNet [3]. We believed that a pretext task would help the models build robust embeddings and weights to improve the downstream performance. We combined a few different image classi-

fication datasets[8][11][17] to create Occluded Urban Images (OccUrban7) dataset for pretraining our models. This dataset contained all the object classes present in the Dhaka set, except for one.

Figure 2: Example of image from OccUrban7 dataset with augmentation and synthetic occlusion.

2.3. Dataset Augmentation

To increase sample size and teach our models to be more robust to the occlusions present in the Dhaka set, we added a variety of artificial occlusions to the pretraining dataset. These treatments simulated the difficulties the model would have to overcome when applied to real situations such as lens flare, motion blur, and occlusion from stacked objects. An example of this pipeline is available in Figure 2 above, featuring a blur effect and a black box occlusion covering up part of the car. We achieved most of these using an Albumentations pipeline paired with a custom Black Box Dropout method and a GAN model trained to apply a fog/overcast effect.

We trained our fog GAN using the CycleGAN[18] architecture, which jointly trains two generators and two discriminators on unpaired datasets (a dataset of foggy street photos[15] and a non-matching dataset of sunny photos). The CycleGAN learns to map between each domain as the first generator models how to apply fog to an image and the second focuses on removing fog. We used a ResNet-style generator so that each block learns a residual connection and preserves the image's low-level structure. The generator only needs to model a small change in the image, rather than entirely recreate it.

2.4. Design Considerations for Our Architecture

We did extensive research in occlusion detection and did a deep-dive on the underlying architecture of Faster R-CNN [10]. The role of the backbone network is crucial for occluded objects where information loss is prevalent. Some of the key challenges for occluded objects is that the context around the occluded object is difficult to understand, the visible parts may be deformed, and scale may affect different objects differently.

We realized the importance of the backbone network as a first step to effective detection. Our design of the custom backbone is inspired by the Bottleneck transformer [13]. The design goal for the backbone network is rich extraction of feature maps from the original image with attention on both features and spatial aspects of the image.

CNNs are good at capturing low-level spatial features, while transformer blocks are known to be effective at capturing complex dependencies in sequential data but are computationally expensive. Our design enhances the best of both worlds and by leveraging a bottle-neck transformer it was computationally feasible to experiment.

Therefore, we designed a layer named "Bottleneck transformer" as it employs a design similar to the botnet blocks in ResNet architecture and combines it with a transformer.

2.5. Descriptions of Backbone Architecture Components

Input projection - A 1x1 convolution (self.conv_proj) used to project the input feature maps.

Transformer block - This layer applies self attention

- For each spatial location in the feature map, the multi-head self-attention mechanism calculates an attention map. This map indicates the importance of every other spatial location in the image when considering the current location.
- The features from all locations are then weighted and combined based on the attention map allowing the model to learn relationships between distant parts and capture both local and global dependencies.
- The results are concatenated and passed through a feed-forward network(MLP) to further process the features under attention and the output is reshaped back to original dimensions.

Lightweight attention mechanism: CBAM:
It focuses on two primary types of attention
Channel Attention - CBAM helps the channel adaptively understand the most important features and suppress the less relevant ones
Spatial Attention - This allows the network to focus on the most informative spatial locations helping to detect critical objects or regions of interest.

2.6. Our Custom Object Detection Architecture: BoTDet

Figure 3 illustrates our custom architecture BoTDet. It utilizes a custom backbone based on both convolutional and attention layers (similar to Bottleneck transformer). The convolutional layers are used to extract feature maps and

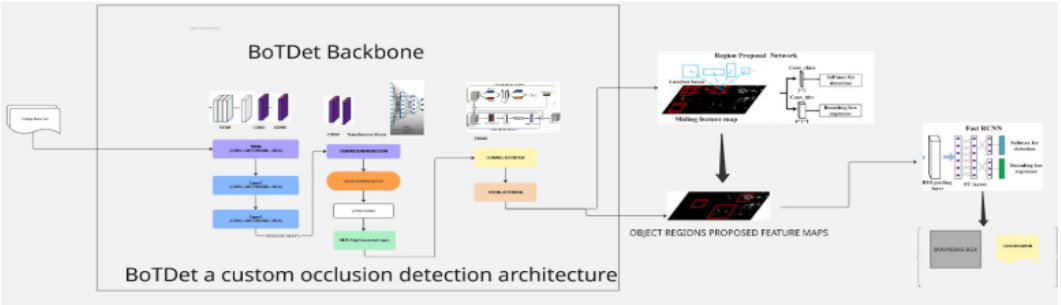


Figure 3: Our Custom Object Detection Architecture: BoTDet

It utilizes a custom backbone based on both convolutional and attention layers (similar to Bottleneck transformer). The convolutional layers are used to extract feature maps and attention mechanisms are leveraged for both channel and spatial dimensions. The feature maps were fed to a region proposal network and classification head similar to that of Faster R-CNN.

Model	Backbone	Epochs	Batch Size	Learning Rate
Faster R-CNN	ResNet50	5	2	0.005
Faster R-CNN	Swin-T	30	4	0.0001
DETR	ResNet50	25	2	0.0001
Pix2Seq	DeiT3 (Encoder)	10	8	0.001
Pix2Seq	ViT (Encoder)	10	8	0.0001
Pix2Seq	ViT (Encoder)	15	8	0.0001
Pix2Seq	Custom Transformer (Encoder)	20	8	0.0001
BoTDet	OccBoTNet	30	8	0.0001

Table 2: Training configurations for object deteciton models

attention mechanisms are leveraged for both channel and spatial dimensions. The feature maps were fed to a region proposal network and classification head similar to that of Faster R-CNN.

3. Experiments and Results

We evaluated the performance of the various models on the Dhaka dataset. The original dataset included augmentations and a split of 4.5k training images, 301 validation images, and 303 test images. The following augmentations were present in the dataset: 50% probability of horizontal flip, random brightness adjustment between -25 and +25%, and random Gaussian blur of between 0 and 1 pixels. Pre-processing of input images from the Dhaka dataset includes transforming the images to tensor and converting the data type of the tensor to floating point values. Most of the models were trained on a NVIDIA V100 GPU (except for Pix2Seq model is trained on V100 GPU). To select the hyperparameters for the best performing models, we evaluated the performance of the model on the validation set after every epoch of training. Table 2 below shows the hyperparameter configurations that gave the best performance for the different models. Besides our custom architecture BoTDet, all other models were fine tuned using pretrained weights. The backbone for BotDet (OccBoTNet) was trained on the OccUrban7 dataset with hyperparameter configurations shown in table 3. We achieved an accuracy of 88.43% on the OccUrban7 dataset using this configuration.

Parameter	Value
learning rate	0.001
weight decay	0.0001
in channels	128
cbam reduction	16
heads	4
mlp ratio	2
batch size	16
epochs	30

Table 3: Training configurations for OccBoTNet (BoTDet backbone)

We used the standard COCO [6] evaluation metric of mean average precision (mAP) to measure the performance of the models. mAP is the mean of average precision (AP) scores across all classes. This is calculated for Intersection

Model	val		test	
	mAP@0.5	mAP@[0.5:0.95]	mAP@0.5	mAP@[0.5:0.95]
Faster R-CNN	0.748	0.454	0.776	0.489
DETR	0.539	0.260	0.539	0.260
BoTDet	0.117	0.054	0.218	0.109

Table 4: Model Performance Comparison

over Union (IoU) thresholds at 0.5 and 0.5:0.95 (0.5 to 0.95 in steps of 0.05). The value for IoU indicates how much overlap is there between the predicted bounding box and the ground truth bounding box, while AP captures how well the model classifies each object. By combining these two metrics, mAP provides a single overall metric that captures how well an object detection model can identify objects. An mAP value of 1 indicates that the model is able to perfectly identify (locate and classify) all objects in the dataset.

For each model, we evaluated its mAP scores on the validation set after training one epoch, and selected those that performed best on the validation set. Table 4 below shows the mAP values that different models obtained from the validation and test set. We were not able to get the Faster R-CNN with the Swin transformer as backbone to learn well on the Dhaka dataset. The learning rate used with the Adam optimizer was tuned between 0.1 to 0.0001. A scheduler with the 1cycle policy was also experimented with to see if introducing warm-up for the Swin transformer would lead to better performance. We hypothesize that more data or a significantly larger number of epochs might be required to train the model with a Swin backbone. As a result, we have decided not to include the Swin transformer in the quantitative analysis below.

Table 5 shows the performance of models based on Pix2seq. We observed that the ViT [4] encoder generally outperformed the DeiT [14] encoder, especially under occlusion. The ViT benefited from self-supervised pretraining on large datasets, enabling it to learn richer global representations and contextual relationships. Unlike DeiT, which is optimized for classification with limited data via supervised learning and distillation tricks, ViT's architecture allows global attention from the first layer, making it more effective at reasoning about partially visible objects. This resulted in stronger robustness to occlusion, better spatial understanding, and superior performance in localization-heavy tasks like object detection.

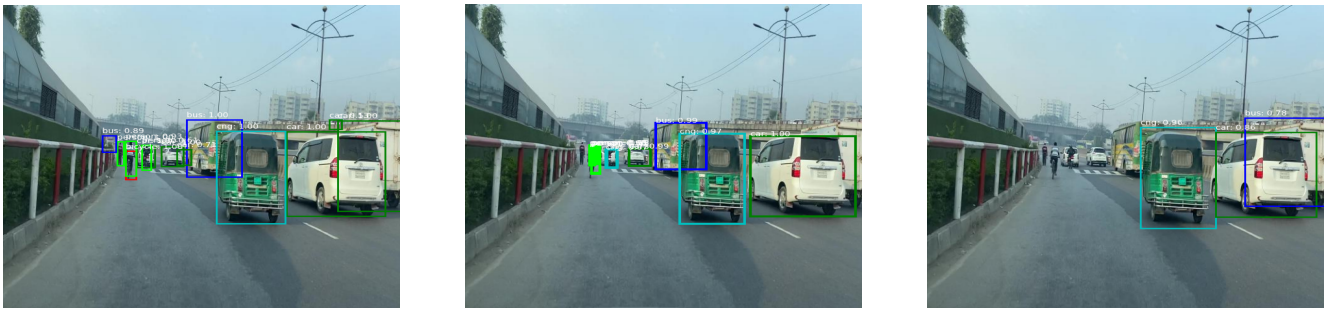
We also trained a custom encoder based on the DeiT architecture with randomly initialized weights, using an augmented dataset containing visually similar classes such as buses, cars, and rickshaws. Our initial hypothesis was that incorporating related object categories would enhance performance. However, the results did not support this assumption. This may be due to the fact that, in heavily occluded scenarios, the contextual relationships learned by the transformer encoder play a limited role, since occlusion often lacks consistent patterns or logical structure from which the model can generalize.

Model	mAP
Pix2Seq (DeiT Encoder)	0.243
Pix2Seq (ViT Encoder)	0.293
Pix2Seq (Custom Encoder)	0.132

Table 5: Model Performance: Pix2Seq

Figure 4 shows visualizations of model predictions side by side on an image in the test set for a qualitative analysis. Notice how this is quite a complex task considering the numerous occlusions and small objects present in the images.

Our custom network architecture BoTDet did not perform well compared to other finetuned models: Faster R-CNN, and DETR. Considering that convolutional neural networks and transformers have different strengths, we believe our architecture which uses both convolutional and attention layers should have inherited the best of both worlds. The convolutions, with its inductive bias towards local features, should enable the model to identify partially occluded objects that have visible parts, while the attention mechanism, which excels at modeling long-range dependencies and global context, should handle more severe occlusions. We believe the reason our architecture did not perform well was due to a lack of data. The other models were trained on large amounts of data before being finetuned. Both Faster R-CNN and DETR, used a ResNet backbone which was trained on ImageNet [3]. The models were also trained on the COCO dataset [6] before being finetuned. Both the OccUrban7 dataset and Dhaka dataset had a significantly smaller amount of data compared to ImageNet and COCO respectively. It is also known that transformers require large amounts of data. Therefore, it is also possible that attention layers were not able to learn well due to the limited amount of data available in our dataset as we suspected with using the Swin transformer as a backbone.



(a) Faster R-CNN output (b) DETR output (c) BoTDet output

Figure 4: Comparison of the output predictions of different models on an image in the test set

4. Discussion

One limitation of this study is the inability to evaluate model performance specifically on occluded objects using occlusion-specific metrics such as Occluded mAP. The dataset used did not include explicit annotations indicating which objects were occluded, and due to time constraints, it was not feasible to manually annotate occluded instances for this purpose. As a result, the evaluation focuses on overall detection performance without isolating the models' effectiveness under occlusion. Another limitation is the limited number of data available in the Dhaka dataset to be used for training architectures based on transformers. Future work to create more data with natural occlusion in outdoor scenes and annotations for occluded objects will help make improvements to this field.

5. Work Division

Summary of contributions are provided in Table 6. Code for the official implementation of DETR from DETR GitHub repository was used and adapted to finetune the model on the Dhaka dataset. To evaluate the performance of our models on standard COCO evaluation metrics, we used helper functions from: PyTorch vision references.

References

[1] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020. 1, 2

[2] Ting Chen, Saurabh Saxena, Lala Li, David J Fleet, and Geoffrey Hinton. Pix2seq: A language modeling framework for object detection. *arXiv preprint arXiv:2109.10852*, 2021. 1, 2

[3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 2, 5

[4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 5

[5] Parvinder Kaur, Baljit Singh Khehra, and Er Bhupinder Singh Mavi. Data augmentation for object detection: A review. In *2021 IEEE International Midwest Symposium on Circuits and Systems (MWSCAS)*, pages 537–543. IEEE, 2021. 1

[6] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer vision–ECCV 2014: 13th European conference, zurich, Switzerland, September 6-12, 2014, proceedings, part v 13*, pages 740–755. Springer, 2014. 2, 4, 5

[7] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021. 1, 2

[8] Mohamed Maher. Vehicle classification. https://www.kaggle.com/datasets/mohamedmaher5/vehicle-classification. Accessed: April 26, 2025. 3

[9] Tanzim Mostafa, Sartaj Jamal Chowdhury, Md Khalilur Rhaman, and Md Golam Rabiul Alam. Occluded object de-

Student Name	Contributed Aspects	Details
Paing	Implementation Dhaka dataset	Wrote PyTorch code to implement the custom dataset.
Paing	Implement finetuning script	Wrote script using PyTorch to finetune models on Dhaka dataset
Paing	Finetune Faster R-CNN & DETR models	Finetuned Faster R-CNN, Faster R-CNN with Swin transformer backbone, and DETR on Dhaka dataset
Shafeeque	Research	Researched multiple papers on occlusion detection and proposed SWIN transformer
Shafeeque	Custom backbone "BotDet"	Designed and Implemented custom backbone network "BotDet"
Shafeeque	Training custom backbone	Trained the custom backbone for classification training on CIFAR-10 and did transfer learning(with Faster-R-CNN) on FUDAN dataset.
Shafeeque	Training custom backbone	Trained the custom backbone for transfer learning(with Faster-R-CNN) on DHAKA dataset.
Shafeeque	Training custom backbone	Trained the custom backbone for classification training on OccUrban7 dataset. Ran out of credits and was assisted by Eli Smith for this training.
Eli	Pretraining dataset construction	Researched and combined multiple public datasets to build a domain-appropriate pretraining dataset for final occlusion detection task.
Eli	Data augmentation pipeline	Developed and implemented an Albumentations pipeline, applying a range of geometric and photometric transforms (flips, color jitter, contrast)
Eli	CycleGAN-based fog model	Trained a CycleGAN model to generate realistic fog overlays on clear-weather training images. Integrated into augmentation pipeline for pretraining.
Chen	Research and implement Pix2seq architecture	Research transformer models, especially pix2seq architecture and write the entire pipeline for implementation
Chen	Pix2Seq Encoder Study (DeiT and ViT)	Implement DeiT and ViT and Train and evaluate both encoders and compare performance
Chen	Train customer Encoder for Pix2Seq	Train a custom encoder using architecture similar to DeiT and compare performance.

Table 6: Contributions of team members.

tection for autonomous vehicles employing yolov5, yolox and faster r-cnn. In *2022 IEEE 13th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, pages 0405–0410. IEEE, 2022. 2

[10] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28, 2015. 1, 2, 3

[11] Vishal B. Sadanand. Deepfashion-1. https://www.kaggle.com/datasets/vishalbsadanand/deepfashion-1. Accessed: April 26, 2025. 3

[12] Guang Shu, Afshin Dehghan, Omar Oreifej, Emily Hand, and Mubarak Shah. Part-based multiple-person tracking with partial occlusion handling. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1815–1821. IEEE, 2012. 1

[13] Aravind Srinivas, Tsung-Yi Lin, Niki Parmar, Jonathon Shlens, Pieter Abbeel, and Ashish Vaswani. Bottleneck transformers for visual recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16519–16529, 2021. 3

[14] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021. 2, 5

[15] Yessica Tuteja. Foggy cityscapes image dataset. https://www.kaggle.com/datasets/yessicatuteja/foggy-cityscapes-image-dataset. Accessed: April 26, 2025. 3

[16] Angtian Wang, Yihong Sun, Adam Kortylewski, and Alan L Yuille. Robust object detection under occlusion with context-aware compositionalnets. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*,

pages 12645–12654, 2020. 1

[17] Yijun Zhai, Pengzhan Zhou, Yuepeng He, Fang Qu, Zhida Qin, Xianlong Jiao, Guiyan Liu, and Songtao Guo. Fedrav: Hierarchically federated region-learning for traffic object classification of autonomous vehicles. *arXiv preprint arXiv:2411.13979*, 2024. 3

[18] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Computer Vision (ICCV), 2017 IEEE International Conference on*, 2017. 3

A. Project Code Repository

The link to our GitHub repository for this project is available here

A.1. BoTDet Model Architecture

```
BotDet(
  (backbone): BoTNetBackbone(
    (stem): Sequential(
      (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (1): BatchNorm2d(64)
      (2): ReLU(inplace=True)
    )
    (layer1): Sequential(
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (1): BatchNorm2d(128)
      (2): ReLU(inplace=True)
    )
    (layer2): Sequential(
      (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (1): BatchNorm2d(128)
      (2): ReLU(inplace=True)
    )
    (botnet): BottleneckTransformerLayer(...)
    (cbam): CBAM(...)
  )
  (rpn): RegionProposalNetwork(...)
  (roi_heads): RoIHeads(...)
)
```